

Rapport technique — Comptabilité financière (Projet Fulltang)

Équipe Fulltang

January 19, 2026

Contents

1 Message d'introduction

A tous le monde, j'attends un document présentant comment vous avez fait vos tâches, le problème que vous avez résolu, comment vous l'avez fait.

Ce document se limite volontairement à la **comptabilité financière** (back + front). Il décrit:

- les modules et classes principales (backend Django + frontend React),
- les workflows métiers (plan comptable, écritures, clôture, rapports, trésorerie, TVA, budget),
- les problèmes rencontrés et la démarche de résolution (build, fusion front/back, mise à jour backend).

2 Périmètre et modules du projet (comptabilité financière uniquement)

2.1 Backend (Django / DRF)

Localisation: ihm_fultang-main/accounting/

- Modèles: `accounting/models_financier.py`
- Serializers: `accounting/serializers.py`
- API Views (DRF): `accounting/api_views/*.py`
- Routage API: `accounting/urls.py`

2.2 Frontend (React / Vite)

Localisation: ihm_fultang-frontend-v1/src/

- Pages Comptabilité: `Pages/AccountantNew/*`
- Services API Comptabilité: `Services/Accounting/*`
- Client HTTP: `Utils/axiosInstancefinancial.js`

3 Problèmes rencontrés et résolutions

3.1 Problème 1 — Build de l'application

- **Problème:** *le build de l'application (résolue grâce à une vraie connexion).*
- **Impact:** build instable / échec lors du téléchargement des dépendances (pip/npm) et/ou lors des étapes d'installation en container.

- **Résolution:** mise en place/usage d'une **connexion réseau fiable** pendant le build (accès registries, DNS, proxy si nécessaire), puis relance des builds Docker.
- **Validation:** build complet reproductible (installation dépendances, compilation assets, démarrage des services).

3.2 Problème 2 — Fusion frontend / backend

- **Problème:** *la fusion frontend backend (resolu en changeant complètement les page du backend pour épouser le nouveau backend).*
- **Contexte:** existence de deux générations d'écrans/services côté front (ex: `Pages/Accountant/` et `Pages/AccountantNew/`) et d'endpoints côté back (anciens vs nouveaux patterns).
- **Résolution:** alignement des pages et du routage afin d'**épouser le nouveau backend**:
 - standardisation des bases d'URL `/api/v1/accounting/*`,
 - refactor des pages "AccountantNew" pour utiliser `Services/Accounting/*`,
 - contrôle d'accès via rôle `Accountant` (dashboards).
- **Validation:** navigation comptable cohérente (plan comptable, journaux/écritures, périodes, rapports) et appels API centralisés.

3.3 Problème 3 — Mise à jour de la comptabilité financière côté backend

- **Problème:** *la mise a jour de la comptabilite financiere cote backend (ajout des models url, serializer view etc).*
- **Résolution:**
 - ajout/extension des **models** comptables (plan comptable, journaux, écritures, tiers, TVA, budget, périodes, trésorerie, immobilisations),
 - exposition des **endpoints** via DRF `ViewSet` et `DefaultRouter`,
 - création/extension des **serializers** avec validations (équilibre débit/crédit, période ouverte, etc.).
- **Validation:** endpoints CRUD + actions métiers (validation d'écritures, rapports, clôture de période) disponibles via `/api/v1/accounting/`.

4 Architecture comptabilité financière — Backend

4.1 Modèles (classes) principales

Les classes ci-dessous constituent le socle de la comptabilité financière (`accounting/models_financier.`

Classe	Rôle métier / principales responsabilités
--------	---

ChartOfAccounts	Plan comptable: code/label, hiérarchie (parent/enfants), type (actif, passif, produits, charges), calcul de solde (mouvements) et sens normal (débit/crédit).
Journal	Journaux comptables (ventes, achats, banque, caisse, général, OD) + comptes par défaut.
JournalEntry	Écriture comptable: date, journal, description, état (DRAFT/POSTED), totaux débit/crédit, numéro de pièce (voucher), validation et mise à jour des soldes.
JournalEntryLine	Lignes d'écriture: compte, libellé, débit/crédit, tiers (client/fournisseur), analytique.
AccountingPeriod	Périodes comptables mensuelles: état (OPEN/CLOSED/LOCKED), clôture, contrôle d'ouverture lors des validations.
AccountPeriodBalance	Soldes par période et par compte: mouvements débit/crédit et soldes d'ouverture/clôture.
Supplier / Customer	Tiers (fournisseurs/clients) rattachés à un compte comptable.
Asset (FixedAsset)	Immobilisations: coût, durée, méthode d'amortissement, comptes (immobilisation, amortissement, charge) + calcul VNC/amortissement.
Inventory	Inventaire/stock: quantités, valorisation, compte de stock associé.
Budget / BudgetLine	Budgets (annuel/trimestriel/mensuel) et ventilation mensuelle par compte (+ analytique optionnel).
VAT	TVA (collectée/déductible) par période, statut et lien possible vers écriture.
TaxRate / TaxDeclaration	Taxes: taux et déclarations (TVA, impôt, charges) avec calcul de pénalités de retard.
BankAccount / BankReconciliation	Trésorerie: comptes bancaires et rapprochements (solde relevé vs comptable, ajustements).
FinancialRatio	Ratios financiers (liquidité, rentabilité, etc.) par période.
AccountingOperation	Lien entre facturation médicale (Bill) et écriture comptable générée.

4.2 Serializers: validation et règles

Localisation: `accounting/serializers.py`

- **Écritures:**

- `JournalEntrySerializer`: impose **au moins 2 lignes** et un **équilibre** (débit = crédit).
- `JournalEntryLineSerializer`: empêche débit et crédit simultanés, exige au moins un montant, interdit les montants négatifs.

- **Périodes:**

- `validate_entry_date`: empêche la saisie sur période non ouverte (ou crée la période si absente).

- **Tiers / trésorerie / taxes:** serializers enrichis avec champs calculés (ex: pénalités, solde ajusté, labels de comptes).

4.3 Endpoints et URLs (DRF Router)

Router: `accounting/urls.py` via `DefaultRouter`.

Endpoint	Fonction
<code>/api/v1/accounting/chart-of-accounts/</code>	CRUD Comptes comptables.
<code>/api/v1/accounting/journals/</code>	CRUD journaux.
<code>/api/v1/accounting/journals/entries/</code>	CRUD écritures + actions métiers (validation, brouillons).
<code>/api/v1/accounting/periods/</code>	Gestion des périodes comptables (listing, clôture).
<code>/api/v1/accounting/reports/</code>	Rapports financiers: bilan, compte de résultat, balance.
<code>/api/v1/accounting/budgets/</code>	Budgets + actions (approbation, totaux).
<code>/api/v1/accounting/bank-accounts/</code>	Comptes bancaires.
<code>/api/v1/accounting/bank-reconciliation/</code>	Réconciliation bancaire.
<code>/api/v1/accounting/vat/</code>	TVA.
<code>/api/v1/accounting/tax-rates/</code> <code>/ /tax-declarations/</code>	Taux et déclarations fiscales.
<code>/api/v1/accounting/suppliers/</code> <code>/ /customers/</code>	Tiers.
<code>/api/v1/accounting/fixed-assets/</code> <code>/ /inventory/ / /payroll/</code>	Immobilisations, stocks, paie (si activés dans le périmètre).
<code>/api/v1/accounting/statistics/</code>	Statistiques globales comptables (dont factures comptabilisées).

5 Workflows Comptabilité financière (Back + Front)

5.1 Workflow 1 — Plan comptable (création/édition/suppression)

Objectif: administrer les comptes comptables (actif/passif/produit/charge) et servir de base à toutes les écritures.

- **Écran front:** `Pages/AccountantNew/Comptabilité de Base/PlanComptable.jsx`
- **Service front:** `Services/Accounting/chartOfAccountsService.js`
- **Endpoint back:** `/api/v1/accounting/chart-of-accounts/`
- **Étapes:**
 - l'utilisateur (rôle **Accountant**) liste/recherche les comptes,
 - crée un compte (code, libellé, classe, type) ou modifie un compte existant,
 - supprime si nécessaire (avec confirmation).

5.2 Workflow 2 — Saisie d’une écriture comptable (brouillon)

Objectif: saisir une opération en écriture **équilibrée** (débit = crédit) avec 2 lignes minimum.

- **Écran front:** Pages/AccountantNew/Comptabilité de Base/JournalsEntries.jsx
- **Service front:** Services/Accounting/journalEntryService.js
- **Endpoint back:** /api/v1/accounting/journal-entries/
- **Étapes:**
 - choix du journal + date + description + référence,
 - ajout de lignes (compte, libellé, débit/crédit),
 - création (état DRAFT) via POST.

5.3 Workflow 3 — Validation d’une écriture (posting)

Objectif: passer une écriture de DRAFT à POSTED en respectant les contrôles:

- période comptable **ouverte**,
- écriture **équilibrée**,
- génération du numéro de pièce (voucher) et **mise à jour des soldes** par période.
- **Action backend:** JournalEntryViewSet.validate_entry (action DRF)
- **URL attendue:** /api/v1/accounting/journal-entries/{id}/validate_entry/
- **Effets:** l’écriture devient POSTED, validated_at est remplie, et AccountPeriodBalance est mis à jour.

5.4 Workflow 4 — Clôture de période et contrôle d’accès

Objectif: empêcher les écritures sur des périodes clôturées/verrouillées.

- **Modèle:** AccountingPeriod (états OPEN/CLOSED/LOCKED).
- **Règle:** JournalEntry.check_period_is_open() interdit la validation si la période n’est pas OPEN.
- **Écran front:** Pages/AccountantNew/Cloture & Reports/Periods.jsx (gestion des périodes).

5.5 Workflow 5 — Rapports financiers (Bilan / Compte de résultat / Balance)

Objectif: donner une visibilité financière à une période (par dates).

- **Écran front:** `Pages/AccountantNew/Cloture & Reports/FinancialReports.jsx`
- **Service front:** `Services/Accounting/financialReportService.js`
- **Endpoints back:**
 - `/api/v1/accounting/reports/balance_sheet/`
 - `/api/v1/accounting/reports/income_statement/`
 - `/api/v1/accounting/reports/trial_balance/`
- **Principe:** agrégation par `ChartOfAccounts.get_balance(start_date, end_date)` sur les écritures validées (POSTED).

5.6 Workflow 6 — Statistiques (factures comptabilisées)

Objectif: suivre les factures médicales et leur statut de comptabilisation.

- **Endpoint back:** `/api/v1/accounting/statistics/`
- **Données:** nombre de factures, factures comptabilisées vs non comptabilisées, filtrage date.

6 Frontend — Écrans “AccountantNew” (cartographie)

But: documenter les workflows comptables visibles par un comptable.

Écran	Service	Endpoint principal
Plan comptable	<code>chartOfAccountsService</code>	<code>/api/v1/accounting/chart-of-accounts/</code>
Journaux/Écritures	<code>journalService</code> , <code>journalEntryService</code>	<code>/api/v1/accounting/journals/</code> , <code>/api/v1/accounting/journal-entries/</code>
Périodes / Clôture	<code>accountingPeriodService</code>	<code>/api/v1/accounting/periods/</code>
Rapports financiers	<code>financialReportService</code>	<code>/api/v1/accounting/reports/*</code>
Budgets	<code>budgetService</code>	<code>/api/v1/accounting/budgets/</code>
Trésorerie (banques)	<code>bankAccountService</code> , <code>bankReconciliationService</code>	<code>/api/v1/accounting/bank-accounts/</code> , <code>/api/v1/accounting/bank-reconciliati</code>
TVA & Fiscalité	<code>vatService</code> , <code>taxService</code>	<code>/api/v1/accounting/vat/</code> , <code>/api/v1/accounting/tax-*</code>
Fournisseurs/Clients	<code>supplierService</code> , <code>customerService</code>	<code>/api/v1/accounting/suppliers/</code> , <code>/api/v1/accounting/customers/</code>
Immobilisations / Stock	<code>fixedAssetService</code> , <code>inventoryService</code>	<code>/api/v1/accounting/fixed-assets/</code> , <code>/api/v1/accounting/inventory/</code>
Paie	<code>payrollService</code>	<code>/api/v1/accounting/payroll/</code>

7 Conclusion

La comptabilité financière du projet Fulltang repose sur:

- un **backend Django/DRF** structuré autour des **écritures comptables** (`JournalEntry` + `JournalEntryLine`) et du **plan comptable** (`ChartOfAccounts`),
- des **contrôles métier** (équilibrage, périodes ouvertes, immutabilité après validation),
- un **frontend React** “AccountantNew” qui couvre les écrans essentiels (plan comptable, écritures, clôture, rapports, trésorerie, TVA, budget).

8 Annexes (à compléter si besoin)

- Captures d’écran des pages “AccountantNew”.
- Exemple d’écriture équilibrée (débit/crédit).
- Exemple de génération d’un Bilan / Compte de résultat sur une période.